Graph Search

I/P :- Undirected / Directed graph. $G = (V, E)$ and a starting vertex s.

Goal :- Identify the vertices of V reachable from s.

Applications

Checking Connectivity

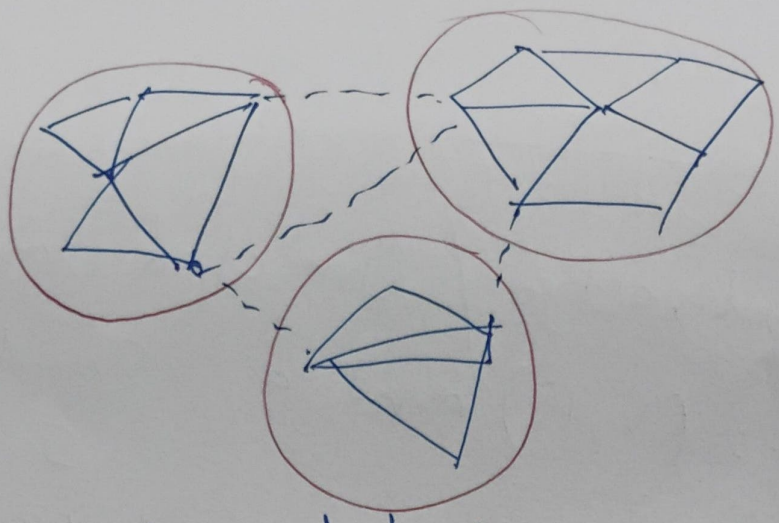
- Web crawling, "friend finder" on Social networks.
- Academic - Erdős no.?

Shortest paths

- Fewest moves to Solve Rubik's Cube?

Revealing Structure

- Finding Community Structure in networks.

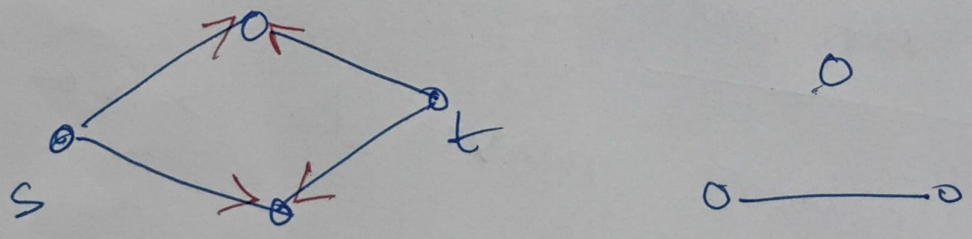


Community detection

Generic graph searching

I/p :- An undirected or directed graph $G = (V, E)$, starting vertex s.

Goal :- Identify the vertices of V reachable from s.



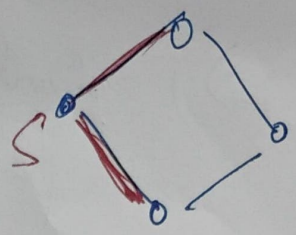
Without direction - Yes.

s to t ? win — No.

[Directed graph reachability is a difficult prb. for parallel computing p.o.v.]

Generic algo.

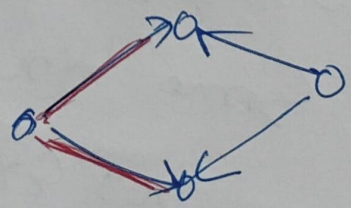
Initially s is Explored, other vertices unexplored.



While some edge $(u,v) \in E$ with u explored and v unexplored.

  mark v Explored.

Directed Cases



Done!

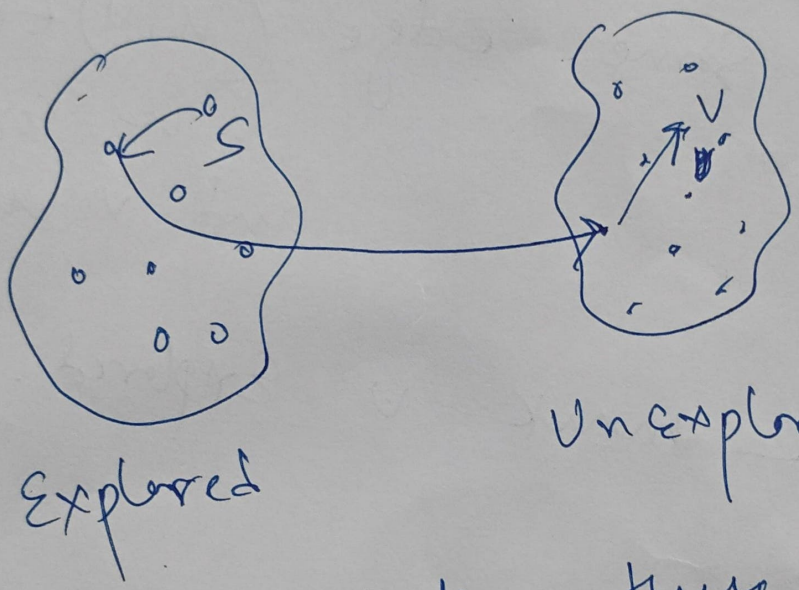
Claim: At the Conclusion of generic algorithm.

v is marked Explored $\Leftrightarrow$ there is a path from s to v.

Proof sketch

($\Rightarrow$) Only way to explore is via paths for s.

($\Leftarrow$) By contradiction



on $s \rightsquigarrow v$ path, there must be some $(x, y) \in E$ s.t

x: explored y: unexplored

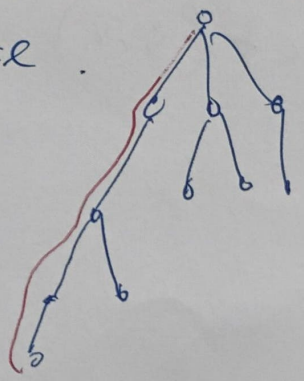
But then the algorithm would not have terminated.

Canonical graph searching

BFS :- Explore nodes in "layers" (e.g. Erdös no.).

— Shortest paths & and Connected components.

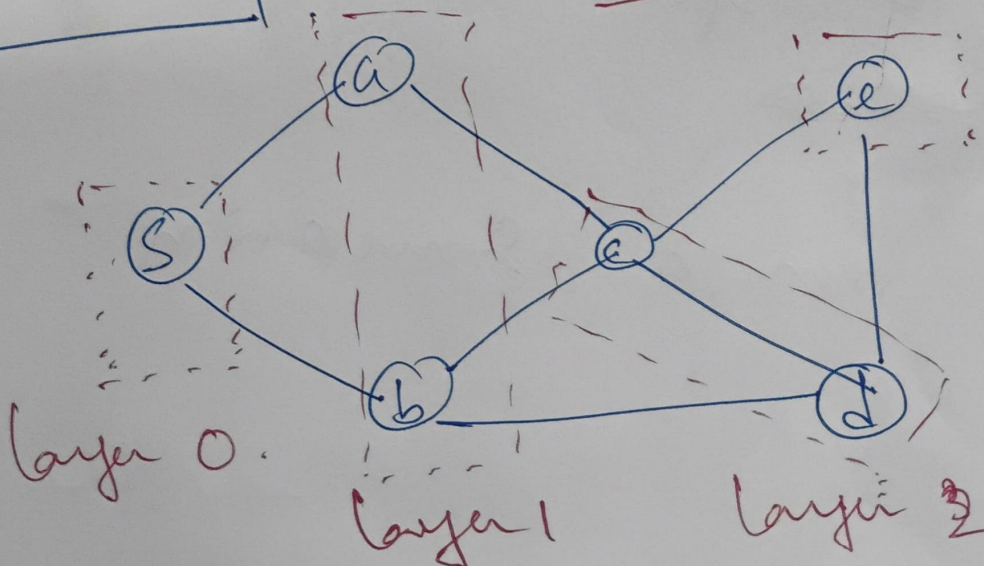
DFS :- Explores aggressively like a maze.



Linear time : $O(m+n)$

Quick Recap

BFS



[Can implement using Queue]

FIFO

Applications.

Shortest paths

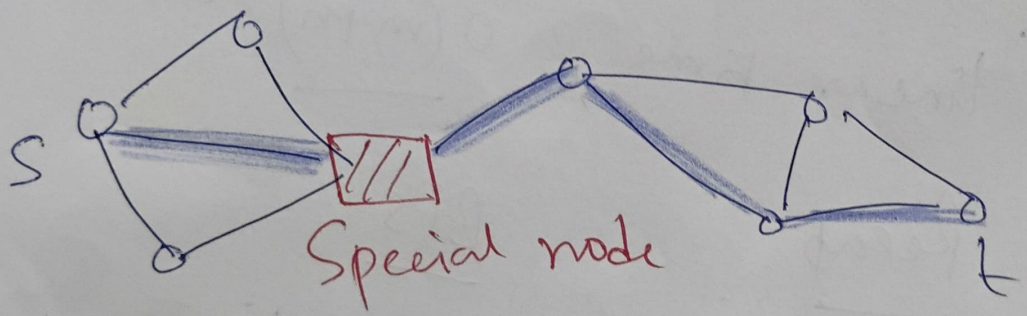
Connected Components

i/p:- An undirected / directed graph $G=(V,E)$ and a starting vertex s.

o/p - dist (s,v) for every vertex $v \in V$.

dist (s,v) = length of shortest path (# of edges) for s to v if one exists

∞, otherwise.



Let's run a simulation.

Recall - BFS template

Mark s as explored, all other vertices unexplored.

Q = a queue data structure (FIFO) initialized with s.

while Q ≠ ∅.

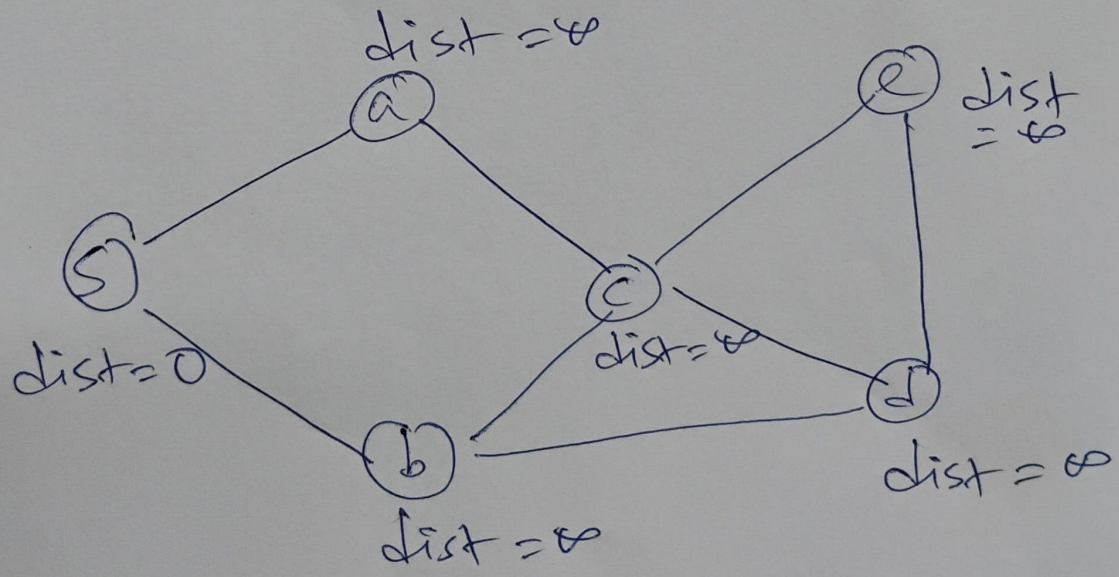
remove the first node of Q, say v.

for each (v, w) ∈ E.

if w is unexplored

mark w as explored.

add w to the end of Q.



Q = s
Step 1

Q = $a
S-2

Q = $ab
S-3

Q = $abc
S-4

s-5 Q = s a b c d

fully Explored

s-6 Q = s a b c d e

s-7 Q = s a b c d e !

Claim :- At termination.

dist(v) = i ⟺ v is in ith layer.

proof = ex.